

# MARIANO MARCOS STATE UNIVERSITY

College of Engineering  
Department of Computer Engineering

**CPE 114 Software Design**

## FINAL PROJECT

*Backend API Application*

|                        |                    |
|------------------------|--------------------|
| <b>Name:</b>           | Walter Jared C. CO |
| <b>Student No.:</b>    | 24-050499          |
| <b>Year / Section:</b> | BSCpE -2A          |
| <b>Date Submitted:</b> | 10/05/2026         |

*(attach page to project repository link)*

## Goal

This project is the final project of CPE 114. You will design, build, and document a complete backend REST API application that solves a real-world problem of your choosing. Unlike the laboratory exercises that guided you step-by-step through a provided structure, this project requires you to make your own architectural and design decisions just as you would in industry.

You are required to use the same technology stack practiced throughout the semester:

- Node.js - runtime environment
- Express.js - web framework and routing
- Sequelize ORM - data modeling and database interaction
- MySQL - relational database

## Choosing Your Problem Domain

Select one problem domain from the list below, or propose your own subject. Your domain determines what resources your API will manage. Every domain must involve at least two related entities with a meaningful real-world relationship between them.

### Domains

| Domain                   | Example Resources               | Suggested Relationship                                               |
|--------------------------|---------------------------------|----------------------------------------------------------------------|
| <b>Library System</b>    | Books, Members, Authors         | Members borrow many Books; Books have one Author                     |
| <b>Clinic / Hospital</b> | Patients, Doctors, Appointments | Patients book Appointments; Doctors handle many Appointments         |
| <b>E-Commerce</b>        | Products, Orders, Customers     | Customers place many Orders; Orders contain many Products            |
| <b>Event Booking</b>     | Events, Attendees, Venues       | Attendees register for many Events; Events are held at one Venue     |
| <b>School Portal</b>     | Students, Subjects, Teachers    | Students enroll in many Subjects; Subjects are taught by one Teacher |
| <b>Inventory System</b>  | Items, Suppliers, Categories    | Items belong to one Category; Suppliers supply many Items            |
| <b>Fitness Tracker</b>   | Users, Workouts, Exercises      | Users log many Workouts; Workouts contain many Exercises             |
| <b>Custom</b>            | Your choice                     | Must have at least one 1-to-many and one many-to-many relationship   |

## Minimum Technical Requirements

Your project must satisfy all requirements in each category below:

### 1) Data Modeling

- Define at least 3 Sequelize models, each mapped to a MySQL table.
- At least one one-to-many relationship (belongsTo / hasMany) must be implemented and used in API responses.
- At least one many-to-many relationship (belongsToMany) through a junction model must be implemented with working endpoints.
- All models must include appropriate data types.

### 2) REST API Endpoints

For each of your main resources, implement the full CRUD set:

| Method | Route Pattern  | Purpose                                                       |
|--------|----------------|---------------------------------------------------------------|
| GET    | /resources     | Return all records                                            |
| GET    | /resources/:id | Return one record by ID; return 404 if not found              |
| POST   | /resources     | Create a record; return 201 on success, 400 for invalid input |
| PUT    | /resources/:id | Update a record; return 404 if not found                      |

|        |                |                                          |
|--------|----------------|------------------------------------------|
| DELETE | /resources/:id | Delete a record; return 404 if not found |
|--------|----------------|------------------------------------------|

Additionally, implement at least two relationship-specific endpoints (e.g., enroll/unenroll, assign/unassign, add/remove nested resource).

### 3) Input Validation and Error Handling

- All POST routes must validate required fields and return 400 with a descriptive JSON error message when fields are missing or invalid.
- All GET/:id, PUT/:id, and DELETE/:id routes must return 404 with a JSON error when the record does not exist.
- Unexpected server errors must return 500 through a global error handler, not expose stack traces to the client.

### 4) Middleware

- A custom logger middleware must log the HTTP method, URL, and timestamp of every incoming request to the console. (You may use [Pino/Winston](#) optionally to implement more advance logging)
- A 404 catch-all handler must return a JSON response for any undefined route.
- A global error handler with exactly 4 parameters must be the last middleware registered.

### 5) Code Organization (MVC Pattern)

- Routes must only define URL paths and delegate logic to controller functions.
- Controller files must contain all business logic and database calls.
- Models must be defined in a dedicated models/ directory.
- Middleware must be in a dedicated middleware/ directory.

### 6) Configuration and Security

- Database credentials must be stored in a .env file and loaded via the dotenv package or so.
- .env must be listed in .gitignore and must not be committed to your repository.
- A .env.example file with placeholder values must be included in the repository.

## Deliverables

Submit a single GitHub repository link containing all of the following:

### 1) Working Application Code

- The full project codebase committed to a GitHub repository.
- The application must start without errors using: node index.js or npm start
- Sequelize must auto-create all tables on startup via sequelize.sync().
- An .env.example file must be included so reviewers can configure the app.
- A README.md must be in the root of the repository (see #3).

### 2) Postman Collection

Export your Postman collection as a JSON file and commit it to the repository at:

```
/docs/postman_collection.json
```

The collection must include one request for every endpoint in your API, organized into folders by resource. Each request must have a short description explaining what it tests.

### 3) README.md Documentation

Your README.md is your API's public-facing documentation. Write it as if another developer who has never seen your project will read it. It must include all of the following sections:

- Project title and a one-paragraph about description of the problem being solved and any discussion about the application (500 words Minimum)
- Tech stack (Node.js, Express.js, Sequelize, MySQL, list the versions)
- Setup instructions: how to clone, install dependencies, configure .env, and start the server
- Database schema: describe each model (table name, columns, data types, constraints)
- Relationship diagram (ER Diagram): a text-based or image diagram showing how entities are related
- API reference: a table of every endpoint listing the method, path, request body (if any), and example response
- Error responses: document all possible error codes and their JSON structures

## Submission

Submit your GitHub repository link using the form <https://forms.gle/EixqpVPCoUjw8tZh7>. Include your full name, student number, and link in the form.

## Grading Rubric

The project is graded out of 100 points across six criteria. Each criterion has four performance levels: Excellent, Satisfactory, Needs Improvement, and Incomplete.

### Criterion 1: Data Modeling (20 pts)

| Level               | Excellent (18–20)                                  | Satisfactory (13–17)                          | Needs Improvement (7–12)                            | Incomplete (0–6)                   | Indicators                                                                                   |
|---------------------|----------------------------------------------------|-----------------------------------------------|-----------------------------------------------------|------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Models</b>       | 3+ models, correct types, proper constraints       | 3 models with minor issues                    | 2 models or many missing constraints                | Fewer than 2 models or app crashes | Each model maps correctly to a MySQL table with appropriate DataTypes and allowNull settings |
| <b>1-to-Many</b>    | Defined and used in GET by ID response via include | Defined but not included in responses         | Association defined with errors                     | Not implemented                    | belongsTo / hasMany correctly declared; associated data appears in relevant endpoints        |
| <b>Many-to-Many</b> | Junction model works; all three endpoints function | Junction model present; minor endpoint issues | Junction model has errors; endpoints partially work | Not implemented                    | belongsToMany through junction model; add/remove/list endpoints work correctly               |

### Criterion 2: API Endpoints (25 pts)

| Level                         | Excellent (23–25)                                   | Satisfactory (17–22)              | Needs Improvement (9–16)                              | Incomplete (0–8)               | Indicators                                                                        |
|-------------------------------|-----------------------------------------------------|-----------------------------------|-------------------------------------------------------|--------------------------------|-----------------------------------------------------------------------------------|
| <b>CRUD</b>                   | All 5 endpoints work; correct status codes for all  | Most work; 1–2 wrong status codes | Endpoints work partially or mostly wrong status codes | 3+ endpoints missing or broken | GET all, GET by ID, POST (201), PUT, DELETE all respond correctly                 |
| <b>Relationship Endpoints</b> | Both endpoints work correctly with proper responses | One works correctly               | Endpoints exist but partially work                    | Not implemented                | At least 2 relationship-specific endpoints (e.g., enroll/list) function correctly |

### Criterion 3: Validation and Error Handling (15 pts)

| Excellent (14–15)                                                                                                                              | Satisfactory (10–13)                                                        | Needs Improvement (5–9)                                    | Incomplete (0–4)                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------|--------------------------------------------------------|
| 400 returned on all invalid POST requests with descriptive messages; 404 on all missing records; global error handler catches 500-level errors | Most validation present; 1–2 cases return wrong codes or expose error stack | Some validation exists but many cases missing or incorrect | Minimal or no error handling; app crashes on bad input |

**Criterion 4: Middleware and Code Organization (15 pts)**

| Excellent (14–15)                                                                                                                                                    | Satisfactory (10–13)                                                          | Needs Improvement (5–9)                                                  | Incomplete (0–4)                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------|------------------------------------------------|
| Logger middleware logs all requests; 404 catch-all returns JSON; global error handler has 4 params; routes have no inline logic; controllers hold all business logic | Middleware present; minor organizational issues (some inline logic in routes) | Logger or 404 handler missing; significant business logic in route files | No custom middleware; all logic in route files |

**Criterion 5: Configuration, Security, and Repository (10 pts)**

| Excellent (9–10)                                                                                                             | Satisfactory (7–8)                                                | Needs Improvement (4–6)                                                 | Incomplete (0–3)                                |
|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------|
| .env used for all credentials; .env not in repo; .env.example included; .gitignore correct; repo is clean and well-organized | Most items present; .env.example missing or minor gitignore issue | .env present but committed to repo; credentials hardcoded in some files | No .env usage; credentials hardcoded throughout |

**Criterion 6: Documentation (15 pts)**

| Excellent (14–15)                                                                                                                                                                                          | Satisfactory (10–13)                                                                                      | Needs Improvement (5–9)                                                                          | Incomplete (0–4)                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|------------------------------------|
| README has all required sections; API reference table is complete and accurate; Postman collection covers every endpoint with descriptions; a developer could set up and use the API using only the README | README present with most sections; Postman collection present but missing a few endpoints or descriptions | README is sparse; Postman collection missing multiple endpoints; setup instructions insufficient | No README or no Postman collection |

**Score Summary**

| Criterion                             | Points      | Score |
|---------------------------------------|-------------|-------|
| 1 Data Modeling                       | /20         |       |
| 2 API Endpoints                       | /25         |       |
| 3 Validation and Error Handling       | /15         |       |
| 4 Middleware and Code Organization    | /15         |       |
| 5 Configuration, Security, Repository | /10         |       |
| 6 Documentation                       | /15         |       |
| <b>TOTAL</b>                          | <b>/100</b> |       |